Property Recommender System

API Reference Documentation

Version 1.0.0

Base URL
`http://0.0.0.0:8000`

Content Type
`application/json`

Overview

The Property Recommender System API is a semantic real estate recommendation engine built with **FastAPI**, powered by **ChromaDB** for vector-based semantic search and **Groq LLM** for natural language understanding and property comparison. It enables intelligent property search, filtering, RAG-based location Q&A, property comparison, and price prediction.

Core Technologies

- FastAPI — high-performance Python web framework
- ChromaDB — vector database for semantic similarity search
- HuggingFace Embeddings — Qwen3-Embedding-0.6B model
- Groq LLM — llama-3.1-8b-instant for natural language processing
- LangChain — orchestration and RAG chain construction
- LangSmith — prompt management and tracing
- XGBoost — price prediction model

Authentication

This API does not require authentication tokens on endpoints. Environment variables must be configured on the server:

- GROQ_API_KEY — required for LLM-powered endpoints
- LANGSMITH_API_KEY — required for prompt management and tracing
- ALLOWED_ORIGINS — comma-separated list of allowed CORS origins (default: http://localhost:3000)

Base URL

```
http://0.0.0.0:8000
```

Response Format

All endpoints return JSON. Paginated list endpoints follow a standard envelope:

Standard Paginated Response

```
{
  "data": [ ... ],         // Array of property objects
  "total": 1540,           // Total matching records
  "page": 1,               // Current page number
  "size": 20,              // Records per page
  "total_pages": 77        // Total number of pages
}
```

Error Handling

All errors return standard HTTP status codes with a JSON body:

Error Response

```
{
  "detail": "Error message or object describing what went wrong"
```

```
}
```

Data Models

PropertyResponse

Returned as items in the data array of paginated property endpoints.

Parameter	Type	Required	Description
`id`	`string`	**Yes**	Unique property identifier
`location`	`string | null`	**No**	Geographic location or area name
`property_name`	`string | null`	**No**	Display name of the property
`description`	`string | null`	**No**	Full property description text
`m2`	`number | null`	**No**	Total area in square meters
`Beds`	`number | null`	**No**	Number of bedrooms
`Baths`	`number | null`	**No**	Number of bathrooms
`payment_plan`	`string | null`	**No**	Payment plan details or terms
`price`	`string | null`	**No**	Formatted price string (e.g. '2,500,000 EGP')
`price_float`	`number | null`	**No**	Numeric price value for filtering/sorting
`property_type`	`string | null`	**No**	Type of property (e.g. Apartment, Villa)
`tag`	`string | null`	**No**	Promotional tag or label
`url_path`	`string | null`	**No**	Relative URL path to the property listing
`cover_image`	`string | null`	**No**	URL of the property cover image
`developer_logo`	`string | null`	**No**	URL of the developer logo

Endpoints

Health & Status

GET `/` — *Root check*

Returns a simple confirmation that the API is running.

Response 200

```
{ "message": "Property Recommender API running" }
```

GET `/ping` — *Liveness probe*

Lightweight liveness check suitable for load balancer health probes.

Response 200

```
{ "ping": "pong" }
```

Properties

GET `/properties` — *List random properties (paginated)*

Returns a shuffled, paginated list of all properties. The shuffle order is deterministic (random_state=42), ensuring consistent pagination across requests.

Query Parameters

Parameter	Type	Required	Description
`page`	`integer`	**No**	Page number, 1-indexed. Default: 1
`size`	`integer`	**No**	Number of records per page. Default: 20

Response 200 — PaginatedPropertyResponse

```
{
  "data": [ { ...PropertyResponse }, ... ],
  "total": 1540,
  "page": 1,
  "size": 20,
  "total_pages": 77
}
```

GET `/filter-options` — *Get filter dropdown values*

Returns all unique values for location and property_type fields in the dataset. Useful for populating UI filter dropdowns. Both lists are returned sorted alphabetically.

Response 200

```
{
  "locations": ["Fifth Settlement", "New Cairo", "Sheikh Zayed", ...],
  "property_types": ["Apartment", "Chalet", "Duplex", "Villa", ...]
}
```

POST `/filter` — *Filter properties*

Applies one or more hard filters on the dataset and returns a paginated result. All filter fields are optional; omitting a field means no filter is applied for that criterion.

Request Body — FilterRequest

Parameter	Type	Required	Description
`location`	`string | null`	**No**	Partial match on location field (case-insensitive)
`property_type`	`string | null`	**No**	Partial match on property_type field (case-insensitive)
`Beds`	`number | null`	**No**	Exact match for beds; if >= 5, matches 5 or more
`Baths`	`number | null`	**No**	Exact match for baths; if >= 5, matches 5 or more

Parameter	Type	Required	Description
`min_price`	`number | null`	**No**	Minimum price_float value (inclusive)
`max_price`	`number | null`	**No**	Maximum price_float value (inclusive)
`page`	`integer`	**No**	Page number. Default: 1
`size`	`integer`	**No**	Records per page. Default: 20
`top_k`	`integer`	**No**	Reserved field (not used in filter). Default: 150

Example Request Body

```
{
  "location": "New Cairo",
  "property_type": "Apartment",
  "Beds": 3,
  "min_price": 1000000,
  "max_price": 5000000,
  "page": 1,
  "size": 20
}
```

Response 200 — PaginatedPropertyResponse

```
{
  "data": [ { ...PropertyResponse }, ... ],
  "total": 84,
  "page": 1,
  "size": 20,
  "total_pages": 5
}
```

Recommendations

POST `/recommend` — *Semantic property recommendations*

The core recommendation engine. Performs **vector similarity search** against the property ChromaDB to find semantically relevant listings, then passes results through the **Groq LLM** to apply any numerical filters inferred from the natural language query (e.g. budget, number of beds).

Pipeline Steps

- Validate that the query string is non-empty
- Run semantic similarity search in ChromaDB (top_k results)
- Extract and normalize property IDs from search results
- Join matched IDs back to the main DataFrame
- Send query + DataFrame column info to LLM to generate a Pandas filter expression
- Execute the generated filter code safely via exec()
- Fall back to unfiltered semantic results if LLM code fails or produces empty results
- Paginate and return the final result set

Request Body — QueryRequest

Parameter	Type	Required	Description
`query`	`string`	**Yes**	Natural language search query (e.g. 'cozy 2-bed apartment under 3M EGP in New Cairo')
`top_k`	`integer`	**No**	Number of candidates to retrieve from vector store. Default: 150
`page`	`integer`	**No**	Page number. Default: 1
`size`	`integer`	**No**	Records per page. Default: 20

Example Request Body

```
{
  "query": "3 bedroom villa with garden in Sheikh Zayed under 5 million EGP",
  "top_k": 150,
  "page": 1,
  "size": 20
}
```

Response 200

```
{
  "data": [ { ...PropertyResponse }, ... ],
  "total": 12,
  "page": 1,
  "size": 20,
  "total_pages": 1,
  "debug": {
    "query": "...",
    "vectorstore_hits": 150,
    "extracted_ids_count": 150,
    "df_matches_after_id_join": 147,
    "llm_generated_code": "df_filtered = ...",
    "llm_filter_result": "OK — 12 rows after filter"
  }
}
```

Note: The `debug` object is always included in the response and provides transparency into pipeline internals. It is safe to ignore in production UIs.

Location Chat (RAG)

POST `/chat/location` — *Ask questions about locations*

A **Retrieval-Augmented Generation (RAG)** endpoint. Retrieves relevant location documents from the `location_information_db` ChromaDB and uses the LLM to answer questions grounded in that context.

Request Body — ChatRequest

Parameter	Type	Required	Description
`question`	`string`	**Yes**	A natural language question about a location (e.g. 'What amenities are near New Cairo?')

Example Request Body

```
{
  "question": "What are the advantages of living in Sheikh Zayed?"
}
```

Response 200

```
{
  "question": "What are the advantages of living in Sheikh Zayed?",
  "answer": "Sheikh Zayed offers excellent connectivity to central Cairo..."
}
```

Response 400

```
{ "detail": "Question cannot be empty." }
```

Property Comparison

POST `/compare` — *Compare two properties*

Compares two properties by their IDs using a multi-context LangChain chain. Property specifications are sourced from the DataFrame; location context is retrieved from the location ChromaDB vectorstore. The LLM then synthesizes both contexts into a structured comparison.

Request Body — CompareRequest

Parameter	Type	Required	Description
id1	string	Yes	ID of the first property to compare
id2	string	Yes	ID of the second property to compare

Example Request Body

```
{
  "id1": "12345",
  "id2": "67890"
}
```

Response 200

```
{
  "id1": "12345",
  "id2": "67890",
  "comparison": "Property 1 (Sunrise Gardens) is a 3-bed, 2-bath apartment in New
Cairo...
                  Property 2 (Palm Hills) is a 4-bed villa in Sheikh Zayed..."
}
```

Price Prediction

POST `/predict-price` — *Predict property price*

Predicts a property price using a trained **XGBoost model**. The model applies log transformations, target encoding of location, and one-hot encoding of property type before inference.

Feature Engineering Pipeline

- Luxury flag: is_luxury = 1 if m2 > 200
- Log transform: m2 = log1p(m2)
- Location encoding: target-encoded using pre-fitted LabelEncoder
- Ratio features: bed_bath_ratio, total_rooms, m2_per_bed
- Location stats: loc_price_mean, loc_price_std derived from training data
- Property type one-hot encoding aligned to training columns
- Final price: expm1(model.predict(...)) to invert log transformation

Request Body — PricePredictionRequest

Parameter	Type	Required	Description
`location`	`string`	**Yes**	Location name (must match a value known to the encoder)
`property_type`	`string`	**Yes**	Type of property (e.g. Apartment, Villa, Duplex)
`m2`	`number`	**Yes**	Total area in square meters
`Beds`	`number`	**Yes**	Number of bedrooms
`Baths`	`number`	**Yes**	Number of bathrooms

Example Request Body

```
{
  "location": "New Cairo",
  "property_type": "Apartment",
  "m2": 150,
  "Beds": 3,
  "Baths": 2
}
```

Response — PricePredictionResponse

Parameter	Type	Required	Description
`location`	`string`	**Yes**	Echo of the input location
`property_type`	`string`	**Yes**	Echo of the input property type
`m2`	`number`	**Yes**	Echo of the input area
`Beds`	`number`	**Yes**	Echo of the input beds
`Baths`	`number`	**Yes**	Echo of the input baths
`predicted_price_egp`	`number`	**Yes**	Predicted price as a float in EGP

Parameter	Type	Required	Description
`predicted_price_formatted`	`string`	**Yes**	Human-readable formatted price (e.g. '3,250,000.00 EGP')

Response 200

```
{
  "location": "New Cairo",
  "property_type": "Apartment",
  "m2": 150,
  "Beds": 3,
  "Baths": 2,
  "predicted_price_egp": 3250000.0,
  "predicted_price_formatted": "3,250,000.00 EGP"
}
```

Response 503 — Models Unavailable

```
{ "detail": "Price prediction models are currently unavailable." }
```

System Architecture

Startup & Asset Loading

On server startup, the lifespan context manager loads all heavy assets into app.state so they are shared across all requests without reloading:

Asset	Details
Properties DataFrame	Loaded from nawy_properties_cleaned.csv with id column forced to string dtype
Embedding Model	HuggingFace Qwen/Qwen3-Embedding-0.6B with trust_remote_code=True
Property VectorDB	ChromaDB persisted at ./property_recommender_db
Location VectorDB	ChromaDB persisted at ./location_information_db
Groq LLM	llama-3.1-8b-instant, temperature=0, max_tokens=1000
XGBoost Model	Loaded from ./price_model/best_xgb_model.joblib
Location Encoder	Loaded from ./price_model/location_encoder.joblib

CORS Configuration

Allowed origins are read from the `ALLOWED_ORIGINS` environment variable as a comma-separated list. If unset, defaults to `http://localhost:3000`. All HTTP methods and headers are permitted for matched origins.

LLM Prompt Management

Prompts are managed externally via **LangSmith** and pulled at runtime using `client.pull_prompt()`. This allows prompt iteration without code changes.

Prompt Key	Used In
`recommend_prompt`	POST /recommend — generates Pandas filter code from query
`property_location_prompt`	POST /chat/location — answers location questions from retrieved context
`compare_multi_context_prompt`	POST /compare — synthesizes property + location context into comparison

Quick Reference

Endpoint Summary

Method	Path	Category	Description
GET	/	Health	API root check
GET	/ping	Health	Liveness probe
GET	/properties	Properties	Paginated random listing
GET	/filter-options	Properties	Filter dropdown values
POST	/filter	Properties	Hard filter properties
POST	/recommend	Search	Semantic recommendation
POST	/chat/location	AI Chat	RAG location Q&A
POST	/compare	AI Chat	Compare two properties
POST	/predict-price	Prediction	XGBoost price prediction

Required Files & Directories

Parameter	Type	Required	Description
nawy_properties_cleaned.csv	CSV file	Yes	Main property dataset with id column
property_recommender_db/	Directory	Yes	ChromaDB for property vector embeddings
location_information_db/	Directory	Yes	ChromaDB for location context documents
price_model/best_xgb_model.joblib	File	Yes	Trained XGBoost price prediction model
price_model/location_encoder.joblib	File	Yes	Fitted location target encoder